

Capítulo 2

Teleoperação

2.1 Introdução

Nesta atividade você irá preparar o seu robô para *teleoperação*. Ao final dela você poderá controlar os motores do seu robô e receber o vídeo capturado pela câmera em uma página web. O vídeo será transmitido via http em codificação *mjpeg*. Os motores serão controlados por um serviço *websocket* rodando no robô, que receberá comandos de uma página web com javascript.

2.2 Recebendo vídeo do robô via browser

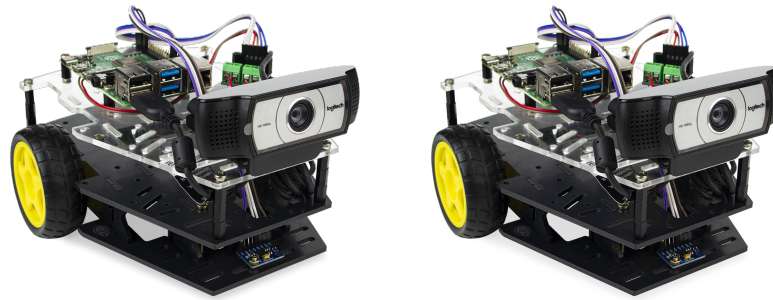
Mas antes, uma breve explicação sobre compactação de vídeo...

2.2.1 Compactação M-JPEG

Um sinal de vídeo puro sem nenhuma compactação usa muita largura de banda para ser transmitido via wifi. É necessário compactá-lo de alguma maneira. A compactação de vídeo, como compactações de mídia em geral, explora redundâncias na forma como o sinal é percebido por humanos. Componentes do sinal pouco perceptíveis são eliminadas do fluxo de dados. Isso significa que esta compactação ocorre *com perdas*. A Figura 2.1 mostra um exemplo de compactação com perdas. Em 2.1a, vê-se a imagem original. Em 2.1b, a imagem compactada pelo algoritmo JPEG. A imagem em 2.1a, armazenada em formato PNG, usa 975 kilobytes. A imagem em 2.1b usa 125 kilobytes. Elas são aparentemente idênticas.

Uma análise mais minuciosa, no entanto revela que são imagens *distintas*. A Figura 2.2 mostra detalhes de texto nas duas imagens. A imagem em 2.2b é claramente diferente da imagem em 2.2a! A diferença está em componentes de alta frequência espacial que não são perceptíveis ao observador humano. Estas componentes são suprimidas da imagem compactada, o que explica a diferença de tamanho.

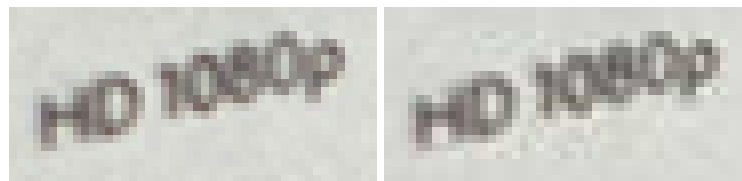
A compactação de vídeo, além de explorar redundâncias *espaciais*, como as do exemplo anterior, explora também redundâncias *temporais*. De fato, na maioria dos sinais de vídeo, um quadro pode ser descrito como uma pequena modificação do quadro anterior. Assim, o algoritmo de compactação codifica



(a) Imagem Original

(b) Imagem Compactada

Figura 2.1: Compressão via JPEG.



(a) Imagem Original

(b) Imagem Compactada

Figura 2.2: Detalhes de compressão.

somente as modificações entre quadros. Mais do que isso, ele considera blocos de quadros, e seleciona a codificação mais eficiente para um determinado bloco.

Infelizmente, este tipo de compactação introduz *latência* na transmissão de vídeo, pois ela requer que todos os quadros de um determinado bloco sejam recebidos antes de iniciar a transmissão do primeiro. Isso faz com que este tipo de compactação seja inapropriada para a teleoperação de um robô, pois o operador precisa responder imediatamente a modificações no ambiente do robô.

Assim, será usada aqui a codificação *M-JPEG*. Nesta codificação, *todos* os quadros são transmitidos integralmente via compressão JPEG. Embora esta codificação use mais largura de banda do que codificações tradicionais com qualidade equivalente, ela apresenta baixíssima latência.

2.2.2 Instalando mjpeg-streamer

A construção do zero de um codificador MJPEG, embora relativamente simples, está fora do escopo de um curso de robótica móvel. Felizmente há um codificador *open-source* apropriado, o *mjpeg-streamer* (disponível em <https://github.com/ArduCAM/mjpg-streamer>).

Siga os passos a seguir para instalar o codificador no seu Raspberry Pi.

Inicialmente, abra um shell no seu Raspberry Pi via *ssh*. Em seguida, instale a ferramenta de gerenciamento de código *git* com o comando:

```
sudo apt install git
```

Agora recupere o código-fonte do *mjpeg-streamer* com o comando:

```
git clone https://github.com/ArduCAM/mjpg-streamer
```

A codificação JPEG do *mjpeg-streamer* é delegada a uma biblioteca externa. Instale a biblioteca de compactação JPEG com o comando:

```
sudo apt install libjpeg62-turbo-dev
```

Do mesmo modo, vamos instalar a biblioteca de desenvolvimento da interface *libcamera*:

```
sudo apt install libcamera-dev
```

O processo de compilação do *mjpeg-streamer* é gerenciado pela ferramenta de configuração *cmake*. Instale-a com o comando:

```
sudo apt install cmake
```

Agora estamos prontos para compilar o código. Primeiro crie um diretório para receber o código compilado com:

```
cd mjpg-streamer/mjpg-streamer-experimental/  
mkdir build  
cd build
```

Compile o código com os comandos:

```
cmake ..  
make
```

Você pode agora receber vídeo ao vivo do seu robô. Teste com o comando:

```
./mjpg_streamer -i "plugins/input_libcamera/input_libcamera.so" -<↵  
o "plugins/output_http/output_http.so -w ../www -p 8008"
```

Você deve receber mensagens como:

Entradas		Saídas	
A	B	A	B
L	L	L	L
H	L	H	L
L	H	L	H
H	H	L	L

Tabela 2.1: Tabela verdade da ponte h l9110.

```

MJPG Streamer Version: git rev: 888↵
    dc3d3990b0092f06da1c95f4928acb16cd7b2
i: Desired Resolution: 640 x 480
Configuring still capture...
Stream configuration adjusted
Still capture setup complete
o: www-folder-path.....: ../www/
o: HTTP TCP port.....: 8008
o: HTTP Listen Address..: (null)
o: username:password....: disabled
o: commands.....: enabled

```

Algumas mensagens de erro do tipo `UVCIOC_CTRL_ADD` são normais e podem ser ignoradas.

Você pode observar o vídeo abrindo o browser na url:

`http://<endereco>:8008/stream_simple.html`

Onde `<endereco>` deve ser substituído pelo endereço/nome de rede do seu Raspberry Pi.

2.3 Acionando os motores

Os motores do robô são acionados por um módulo de ponte h como visto na seção 1.2.4. A ponte h permite o acionamento do motor em ambos os sentidos. Se os motores do kit receberem alimentação de 5V no cabo *vermelho* e 0V no cabo *preto*, vão girar no sentido *anti-horário* (se visto pela roda). Se receberem 5V no cabo *preto* e 0V no cabo *vermelho*, vão girar no sentido *horário*.

Pelas instruções da seção 1.2.4, o cabo preto do motor A está ligado à saída “B” da sua ponte h, enquanto que o cabo vermelho está ligado à saída “A”. Note também que o motor B usa a convenção *inversa* (por quê?).

As saídas da ponte h são controladas por duas entradas (também chamadas “A” e “B”). A tabela verdade está reproduzida na tabela 2.1.

As entradas da ponte h, por sua vez, são controladas por pinos de I/O da Raspberry Pi. Como orientado na tabela 1.2, as entradas “A” dos motor “A” e “B” são controladas pelos pinos 29 e 31, respectivamente, enquanto que as entradas “B” são controladas pelos pinos 32 e 33. Na seção 1.6.2 você controlou os pinos 29 e 31 (GPIO 5 e 6) pelo sistema de arquivos para acionar os motores “A” e “B”. Quando levados para nível “Alto”, acionam os motores via entradas “B”, visto que as correspondentes entradas “A” estão em nível “Baixo”.

Os pinos 32 e 33, conectados às entradas “A”, são especiais. Estes pinos estão internamente conectados a geradores de sinais com modulação por largura de

pulso. Isso significa que o sinal saído por estes oscila rapidamente entre “Alto” e “Baixo”, e a fração do ciclo que o sinal passa em cada ciclo pode ser configurada.

No momento¹, no entanto, todas as soluções para acessar esse hardware em Python envolvem alguma limitação. Há por um lado, a biblioteca `pigpio`, que já vem pré-instalada no Raspberry Pi. Esta é uma ferramenta bastante poderosa, que permite, entre outras coisas, controlar os geradores de pwm em hardware do Raspberry Pi. De fato, ela permite isso e muito mais, inclusive através de uma interface de rede. No entanto, ela requer a execução de um programa separado, o `pipiod`, e os diversos recursos adicionais da biblioteca, tais como transparência em rede, são desnecessários e adicionam complexidades adicionais.

Por outro lado, a biblioteca `rpi_hardware_pwm` é o extremo oposto. É uma biblioteca extremamente simples, que simplesmente controla os geradores de PWM. Embora possa parecer a solução ideal, há um problema: Esta biblioteca não vem pré-instalada e tampouco é instalável pelo gerenciador de pacotes do Raspberry Pi OS.

Vamos usar neste curso a biblioteca `rpi_hardware_pwm`. Se você tem experiência prévia com `pipiod`, pode usá-la para executar as mesmas tarefas.

2.3.1 Instalando pacotes via pip e Ambientes Virtuais Python

Como mencionado, você não vai conseguir instalar `rpi_hardware_pwm` usando o gerenciador `apt` de pacotes do Raspberry Pi OS. Podemos instalá-lo no entanto usando o gerenciador de pacotes Python `pip`. Há um novo dilema aqui no entanto. Como a sua instalação lida com a ação combinada de dois gerenciadores de pacotes simultâneos (`apt` e `pip`)? A resposta é potencialmente mal. Versões conflitantes de pacotes e suas interdependências podem fazer com que aplicativos inteiros deixem de funcionar. Honestamente, este dificilmente seria o caso para um pacote trivial como `rpi_hardware_pwm`, e para essas ocasiões o gerenciador `pip` oferece a opção de nome sugestivo `--break-system-packages` (!).

Vamos aproveitar a ocasião no entanto para demonstrar uma alternativa mais robusta, o uso de *ambientes virtuais*. Um *ambiente virtual* é um espaço de pacotes python isolado do restante do sistema. Então, embora conflitos ainda possam ocorrer, eles estarão restritos ao ambiente virtual e não afetarão demais aplicativos fora deste.

Você precisa especificar um diretório para criar o ambiente virtual. Nesta atividade, vamos presumir que será usado um subdiretório da pasta inicial do usuário chamado `robotvenv`. Abra um novo shell e digite o comando:

```
python -m venv robotvenv
```

Verifique o conteúdo com o comando:

```
ls robotvenv
```

você deve ver algo como:

```
bin  include  lib  lib64  pyenvv.cfg
```

O ambiente precisa ser *ativado*. Faça-o com o comando

```
. robotvenv/bin/activate
```

¹Ano 2024

Veja que o prompt de comando mudou, mostrando agora o nome do ambiente virtual.

Para sair do ambiente virtual, uma vez dentro dele, execute o comando:

```
deactivate
```

2.4 Interfaces Python para GPIO e PWM

Agora você pode instalar o pacote `rpi-hardware-pwm` sem se preocupar com conflitos com outros pacotes. O comando para instalá-lo é:

```
pip install rpi-hardware-pwm
```

Você pode sair do ambiente virtual com o comando `exit`, mas não o faça agora. Vamos usar este pacote para acionar os geradores de PWM e acionar as rodas do robô.

Vamos usar o interpretador python em modo interativo para testar a ponte h. Para tanto coloque antes o robô em um suporte que mantenha as rodas *fora* do solo!

A seguir, abra um interpretador python com o comando:

```
python
```

Carregue a biblioteca `rpi-hardware-pwm` com o comando:

```
from rpi_hardware_pwm import HardwarePWM
```

Relembrando as conexões, o canal de entrada “A” do motor do lado direito está ligado ao gerador de PWM 0. O canal de entrada “A” do motor do lado esquerdo está ligado ao gerador de PWM 1.

Crie objetos com a interface PWM com os comandos:

```
a = HardwarePWM(pwm_channel=0, hz=20, chip=0)
b = HardwarePWM(pwm_channel=1, hz=20, chip=0)
```

Isso configura o sinal de PWM dos pinos 32 e 33 na frequência de 20Hz. Ative os sinais de pwm com o comando

```
a.start(0)
b.start(0)
```

Agora você pode acionar os motores com pwm.

Ative o motor “A” (do lado direito) com 50% do ciclo com o comando:

```
a.change_duty_cycle(50)
```

O motor direito deve girar “para frente”. Você pode parar o motor com o comando:

```
a.change_duty_cycle(0)
```

Experimente acionar o motor em vários níveis de ciclo de pwm, passando 0, 25, 50, 75 e 100 para os parâmetros em `change_duty_cycle`. Faça o mesmo com o motor “B” (do lado esquerdo). Note que ambos os motores giram no sentido de fazer o robô andar “para frente”. Note também que quanto maior o valor passado a `change_duty_cycle`, maior a velocidade do motor.

Agora vamos mudar o sentido de rotação dos motores, alterando os sinais enviados às portas “B” de ambos os motores. Para tanto, vamos novamente usar uma outra biblioteca que vai precisar ser instalada, a `gpiod`.

Se você tem alguma experiência prévia com Raspberry Pi e GPIO, talvez esteja aborrecido com esse texto. Afinal, existe uma biblioteca pré-instalada, a `RPi.GPIO`, perfeitamente capaz de acessar os pinos de GPIO. Pior ainda, ela oferece acionamento de PWM nos pinos! Afinal, o que estamos fazendo aqui, instalando ambientes virtuais e bibliotecas como `gpiod` e `rpi-hardware-pwm`².? A resposta é que estamos passando por um momento de transição na plataforma Raspberry Pi. A interface antiga de GPIO, baseada em arquivos, foi marcada como *obsoleta*. É desta interface que depende `RPi.GPIO`. Ademais, a geração de sinais PWM desta é feita via *software*, e não usa o hardware dedicado. A nova interface, a mesma usada pelas ferramentas `gpiozero` e `gpiomon`, é acessada pela biblioteca `gpiod`. Esperamos que em um futuro próximo esta biblioteca venha pré-instalada com o Raspberry Pi.

Por enquanto, saia do ambiente python com o comando `quit()`.

A biblioteca `gpiod` em sua versão atual³ depende de um componente a ser *compilado* localmente. Para tanto, são necessárias as bibliotecas de desenvolvimento de python. Instale-as com o seguinte comando:

```
sudo apt install python3-dev
```

Instale a seguir o pacote python `gpiod` com o comando (certifique-se de estar dentro do ambiente virtual!):

```
pip install gpiod
```

Volte ao interpretador Python. A biblioteca `gpiod` trabalha com *Gerenciadores de Contexto*, que podem garantir que o estado dos GPIOs é restaurado sempre que os seus objetos são liberados. Isso é *particularmente* útil na nossa aplicação, pois evita, por exemplo, que o seu robô saia em disparada se um erro causar o término do seu programa. Vamos explorar estes gerenciadores mais adiante. Por hora vamos usar `gpiod` de forma mais convencional.

Importe a biblioteca e os símbolos necessários com os comandos:

```
import gpiod
from gpiod.line import Direction, Value
```

A biblioteca `gpiod`, pelas razões de Gerenciamento de Contexto mencionadas anteriormente, requer que você especifique de forma explícita as linhas de GPIO que se deseja acessar, e qual a configuração inicial destas.

Crie uma configuração de *output* com valor inicial “baixo” com o comando:

```
confoutlow = gpiod.LineSettings(direction=Direction.OUTPUT, ↵
    output_value = Value.INACTIVE)
```

Agora vamos acessar o chip “0” (Como nos testes da seção 1.6.2) e solicitar acesso às linhas de GPIO 5 e 6 com essa configuração inicial:

```
chip0 = gpiod.Chip("/dev/gpiochip0")
linhas = chip0.request_lines({5:confoutlow, 6:confoutlow})
```

Finalmente⁴, podemos mudar os níveis do GPIO 5 e 6. Vamos mudá-los para o nível “alto” com os comandos:

```
linhas.set_value(5, Value.ACTIVE)
linhas.set_value(6, Value.ACTIVE)
```

²Pior ainda... a biblioteca `gpiozero` tem interfaces prontas para controle de *motores* por PWM, mas depende de `pipiod`...

³abril-2024

⁴Sim, a interface de `RPi.GPIO` é muito mais simples...

Note que *ambos* os motores imediatamente começam a girar no sentido de fazer o robô *recuar*. Experimente agora repetir os comandos de acionamento do PWM modificando o sinal enviado às entradas “A” de cada motor com `change_duty_cycle` passando valores de 0, 25, 50, 75 e 100. Tente descobrir que valores fazem os motores pararem. E agora? Qual o efeito de se aumentar o valor passado a `change_duty_cycle` sobre a rotação dos motores? Lendo a tabela 2.1, você consegue explicar este comportamento?

Encerre os testes com os comandos:

```
linhas.set_value(5, Value.INACTIVE)
linhas.set_value(5, Value.INACTIVE)
a.stop()
b.stop()
linhas.release()
quit()
```

2.5 Controlando o robô pela web

Agora que você sabe controlar a rotação dos motores usando as bibliotecas `rpi_hardware_pwm` e `gpiod`, vamos usar este conhecimento para finalizar uma aplicação web de operação remota do robô.

Vamos criar uma página web com dois joysticks virtuais (adaptados do projeto <https://github.com/bobboteck/JoyStick>). Estes joysticks virtuais, escritos em javascript, vão enviar comandos para o robô pelo protocolo *websocket*. Recupere o esqueleto da aplicação com o comando:

```
cd ~
git clone https://gitlab.uspdigital.usp.br/thiago/pmr3502-telepresenca.git
```

Você vai precisar também do pacote Python `aiohttp`. Instale-o com o comando (certifique-se de ativar o ambiente virtual):

```
pip install aiohttp
```

Abra um *novo* terminal e ative o servidor `mjpeg` como instruído na seção 2.2.2. Enquanto o servidor `mjpeg` estiver ativo, execute o servidor *dentro do ambiente virtual* com os comandos:

```
cd pmr3502-telepresenca
./servidor.py
```

Abra em um web browser o endereço `http://<endereco>:8080`, onde `<endereco>` deve ser substituído pelo endereço/nome de rede do seu Raspberry Pi.

Note que a página mostra no topo o vídeo do robô e dois joysticks virtuais.

Sugestão: Use um dispositivo com tela sensível ao toque, como um telefone celular, para controlar os dois joysticks ao mesmo tempo.

Veja que a saída do servidor mostra valores de `l` e `r` correspondentes às posições dos joysticks esquerdo e direito respectivamente.

Interrompa o servidor com a combinação `ctrl+c`. Abra o arquivo `motorctrl.py` no editor de sua preferência.



Sugestão O ambiente KDE do laboratório tem um gerenciador de arquivos que pode acessar o seu Raspberry Pi via ssh. Digite na barra de endereços do Dolphin fish://<endereço>, onde <endereço> é o endereço de rede de seu Raspberry Pi. Você pode abrir arquivos de textos no Raspberry Pi com o editor Kate no computador do laboratório

A listagem do arquivo se segue:

```
import gpiod
from gpiod.line import Direction, Value
from rpi_hardware_pwm import HardwarePWM
import contextlib

# Cria um controlador de motor.
# Este gerador é decorado com contextmanager para garantir
# que o estado do gpio e pwm seja resetado na saída de escopo
@contextlib.contextmanager
def cria_controle_motor():
    # Configuração inicial: output com nível inicial baixo
    print("Criando controle de motores...", end="")
    confoutlow = gpiod.LineSettings(direction=Direction.OUTPUT,
    output_value = Value.INACTIVE)
    with gpiod.Chip("/dev/gpiochip0") as chip0:
        # Cria acesso às linhas de GPIO 5 e 6
        with chip0.request_lines({5:confoutlow, 6:confoutlow}) as
linhas:
        pwma = HardwarePWM(0, 20, 0)
        pwmb = HardwarePWM(1, 20, 0)
        motor = motorCtrl(linhas, 5, 6, pwma, pwmb)
        print("Feito.")
        try:
            yield motor
        finally:
            print("Encerrando controle de motores...", end="")
            pwma.stop()
            pwmb.stop()
    print("Feito.")

class motorCtrl():
    def __init__(self, linhas, canal_direita, canal_esquerda,
pwma, pwmb):
        # Acesso a linhas gpio por gpiod
        self._linhas = linhas
        # Numero do canal de GPIO da fase "B" do motor da direita
        self._cdireita = canal_direita
        # Numero do canal de GPIO da fase "B" do motor da
esquerda
        self._cesquerda = canal_esquerda
        # PWM conectado à fase "A" do motor da direita
        self._pwma = pwma
        # PWM conectado à fase "B" do motor da direita
        self._pwmb = pwmb
        self._pwma.start(0)
        self._pwmb.start(0)

    def set_lr(self, l, r):
        # Aplica comandos nos motores:
        # l é o valor (de -100 a 100) para o motor esquerdo
        # r é o valor (de -100 a 100) para o motor esquerdo
```

```
print("motorCtrl.set_lr(l={}, r={})".format(l, r))
```

Você deve reimplementar o método `set_lr` para processar os comandos dos joysticks virtuais. Use o que aprendeu na seção 2.3.

Quando estiver confiante de que consegue controlar ambos os motores pela página, desligue o Raspberry Pi, retire-o da fonte USB e conecte-o ao Power Pack. Coloque o robô no chão, abra novamente os servidores mjpeg e de teleoperação (use dois shells, ou adicione um caractere `&` ao final da invocação do servidor mjpeg) e tente operar remotamente seu robô. Tente navegar pela sala olhando somente a câmera!